



LIONSDEV

Curso de Programação

MÓDULO 6

Sistema de Cadastro e Busca



Professor Nicolas Cardoso Motta

Revisando o Módulo 5



Módulo = arquivo

Cada arquivo cuida de uma coisa e exporta só o que os outros precisam.



CommonJS x ESM

`require` x `import`. O ESM precisa de `"type": "module"` no `package.json`.



Padrão x nomeado

`export default` importa com qualquer nome; o nomeado usa chaves e o nome exato.

Pergunta para abrir: por que `import soma from './soma'`, sem o `.js`, dá erro no ESM?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



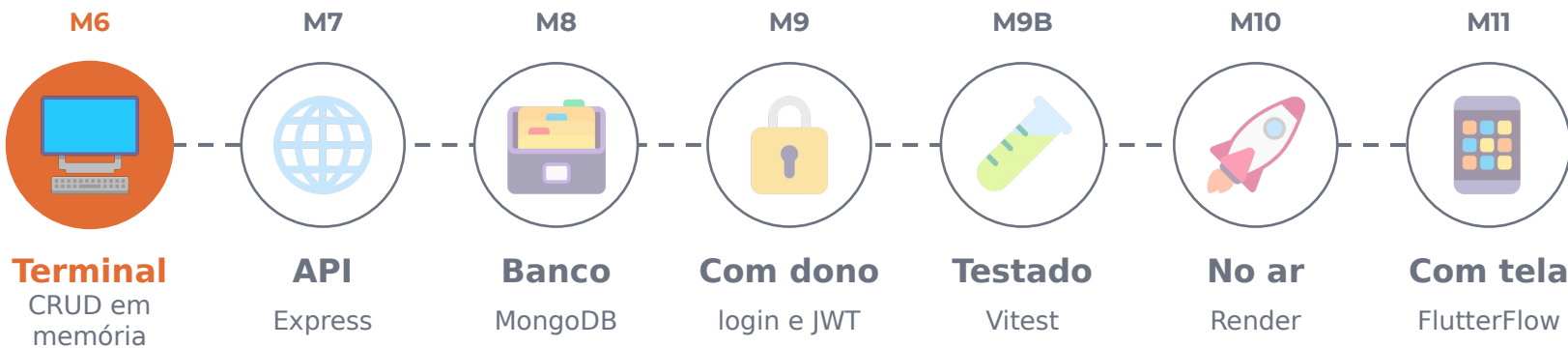
BACKEND



FRONTEND E PROJETO

Você está aqui: **Módulo 6**, Sistema de Cadastro e Busca

O projeto flashcards



NO MÓDULO 6

O flashcards nasce aqui: baralhos e cartões num array, com o mesmo CRUD e a mesma busca da agenda. Na atividade, escolha a lista Flashcards e comece o projeto que vai até o Módulo 11.



Objetivo

- **Compreender e aplicar o padrão CRUD.**
- **Gerenciar dados em memória com arrays e objetos.**
- **Aplicar a modularização do módulo anterior.**
- **Dominar busca e filtragem com `.findIndex()`, `.find()` e `.filter()`.**
- **Relacionar dados de listas diferentes usando IDs.**

Gerenciamento de dados



Imagine uma loja que precisa cadastrar os seus clientes.

Quais requisitos são necessários para criar um sistema desses?

Os quatro requisitos de qualquer cadastro



Cadastro

Criar novos registros.



Listagem

Ver os registros que existem.



Atualização

Alterar os dados de um registro.



Remoção

Apagar um registro.

CRUD: as quatro operações

Quase todo sistema que gerencia dados faz essas quatro operações:

Create · Criar

Cadastrar um novo registro.

```
array.push(novo)
```

Read · Ler

Listar e buscar registros.

```
forEach / find / filter
```

Update · Atualizar

Alterar dados de um registro.

```
findIndex + atribuição
```

Delete · Deletar

Remover um registro.

```
findIndex + splice
```

**Vamos a um exemplo: uma agenda de
contatos!**

Requisitos: cadastro e busca



Cadastro de contato

- Gerar um ID sequencial para cada contato.
- Guardar nome, e-mail e uma lista de telefones.
- Não permitir e-mail duplicado e mostrar uma mensagem de erro.



Busca de contatos

- Listar todos os contatos com as informações cadastradas.
- Mostrar os telefones como uma lista associada ao contato.

Requisitos: atualização e remoção



Atualização de contato

- Atualizar o nome, o e-mail ou os telefones.
- O novo e-mail não pode estar em uso: se estiver, mostrar erro.



Remoção de contato

- Remover um contato informando o ID.
- Pedir confirmação antes de remover.

A estrutura do projeto

```
agenda/
├── app.js           (o menu)
├── contatos.js      (os dados)
├── adicionar_contato.js
├── listar_contatos.js
├── atualizar_contato.js
└── remover_contato.js
```

app.js → o menu: importa tudo e chama cada operação.

contatos.js → a nossa "base de dados": um array de objetos.

uma operação → cada arquivo cuida de uma letra do CRUD. É a modularização do Módulo 5 na prática.

A base de dados: contatos.js

```
const contatos = [  
  {  
    id: 1,  
    nome: 'Ana Silva',  
    telefones: ['(11) 91234-5678'],  
    email: 'anasilva@gmail.com',  
  },  
  {  
    id: 2,  
    nome: 'Nicolas Motta',  
    telefones: ['(11) 93215-6789'],  
    email: 'nicolasmotta@gmail.com',  
  },  
]  
  
export default contatos
```

array de objetos → cada contato é um objeto; todos ficam num array.

telefones → um array dentro do objeto: um contato pode ter vários.

export default → para o menu e as outras operações enxergarem a lista.

banco de dados → num sistema real, usaríamos MongoDB (Módulo 8). Aqui, simulamos na memória.

O menu: app.js

```
import promptSync from 'prompt-sync'
import contatos from './contatos.js'
import listarContatos from './listar_contatos.js'
// ... import das outras três operações

const prompt = promptSync()
let opcao = '0'

while (opcao !== '5') {
  mainMenu() // mostra as opções de 1 a 5
  opcao = prompt('Escolha uma opção: ')
  switch (opcao) {
    case '1': listarContatos(contatos); break
    // ... um case para cada operação
  }
}
```

Aqui a opção fica como **texto**, sem `Number()`: por isso os cases usam aspas (case "1"). O importante é o tipo bater com o do switch.

Por que todos os arquivos enxergam a mesma lista?



O módulo carrega uma vez só

Todo arquivo que importa `contatos.js` recebe o mesmo array, não uma cópia.



Funções recebem a lista

O menu passa `contatos` para cada função. Um `push` ou `splice` lá dentro altera a lista que o menu vê.



`const` não impede mudar o conteúdo

Por isso usamos `push` e `splice`, e nunca `contatos = [...]` (lembra do `const` com array do Módulo 2?).



Create: adicionando um contato

Validando o e-mail antes de salvar

```
function adicionarContato(contatos, novoContato) {  
  let emailExiste = false  
  for (let i = 0; i < contatos.length; i++) {  
    if (contatos[i].email === novoContato.email) {  
      emailExiste = true  
      break    // para a busca assim que encontrar  
    }  
  }  
  
  if (emailExiste) {  
    console.log('Erro: Este e-mail já está cadastrado!')  
    return false  
  }  
  // ... continua no próximo slide
```

O laço percorre a lista procurando o e-mail. Se achar, avisa e devolve false para o menu: nada é salvo.

Gerando o ID e inserindo

```
// gera o ID sequencial: último ID + 1
if (contatos.length > 0) {
  let ultimoContato = contatos[contatos.length - 1]
  novoContato.id = ultimoContato.id + 1
} else {
  novoContato.id = 1 // lista vazia: é o primeiro
}

contatos.push(novoContato)
return true
}

export default adicionarContato
```

O push insere o objeto no fim do array, e o return true avisa o menu que deu certo.

No menu: vários telefones

```
case '2': {  
  let novoContato = {  
    nome: prompt('Nome: '),  
    email: prompt('Email: '),  
    telefones: [],  
  }  
  let addMais = 'sim'  
  while (addMais.toLowerCase() === 'sim') {  
    novoContato.telefones.push(prompt('Telefone: '))  
    addMais = prompt('Adicionar outro telefone? (sim/nao): ')  
  }  
  if (adicionarContato(contatos, novoContato)) {  
    console.log('Contato adicionado com sucesso!')  
  }  
  break  
}
```

O array telefones começa vazio e cresce com push enquanto o usuário responder "sim".



Cuidado: o ID pode se repetir

Com "último ID + 1", remover o **último** contato faz o próximo cadastro **reaproveitar** o ID dele.

- IDs 1, 2 e 3 → remove o 3 → o novo contato também ganha o 3.
- Se outra lista guardava o ID 3 (como uma consulta na clínica), agora ela aponta para a pessoa errada.
- Solução: um contador separado, que só cresce e nunca volta.

```
let proximoId = 4           // começa depois do último ID  
  
novoContato.id = proximoId  
proximoId++
```



Cuidado: Ana@Gmail.com e ana@gmail.com

A comparação diferencia maiúsculas: o **mesmo e-mail** passa duas vezes pela validação.

- Padronize o e-mail antes de comparar e antes de salvar.
- O `trim()` ainda remove os espaços que o usuário digitou sem querer.

```
'Ana@Gmail.com' === 'ana@gmail.com'           // false  
'Ana@Gmail.com'.trim().toLowerCase() === 'ana@gmail.com' // true
```



Read: listando os contatos

listarContatos

```
function listarContatos(contatos) {  
  if (contatos.length === 0) {  
    console.log('Nenhum contato cadastrado.')  
    return  
  }  
  
  contatos.forEach((contato) => {  
    console.log(`\nID: ${contato.id}`)  
    console.log(`Nome: ${contato.nome}`)  
    console.log(`Email: ${contato.email}`)  
    console.log(`Telefones: ${contato.telefones.join(' | ')}`)  
  })  
}
```

Primeiro confere se a lista está vazia: se estiver, avisa e o return encerra a função ali mesmo.

.join(): de array para texto

```
let telefones = ['(11) 91234-5678', '(11) 98765-4321']  
  
console.log(telefones)  
// [ '(11) 91234-5678', '(11) 98765-4321' ]  
  
console.log(telefones.join(' | '))  
// (11) 91234-5678 | (11) 98765-4321
```

O join cola os itens do array numa única string, com o separador que você escolher.

Listar tudo x buscar específico



Listar tudo

Devolve a base inteira. É o que o `listarContatos` faz.



Buscar específico

Devolve só o que atende a um critério: um ID, um nome, um médico.

Mais adiante, vamos usar o `filter` e o `find` para as buscas específicas.



Update: atualizando um contato

.findIndex(): em que posição está?

```
let contatos = [  
  { id: 1, nome: 'Lucas' },  
  { id: 2, nome: 'Vitoria' },  
]  
  
contatos.findIndex((contato) => contato.nome === 'Vitoria') // 1  
contatos.findIndex((contato) => contato.nome === 'Pedro')   // -1
```

Se encontrar, devolve a posição (0, 1, 2...). Se não encontrar, devolve -1, porque 0 já é uma posição válida.

atualizarContato: encontrando o contato

```
function atualizarContato(contatos, id, novosDados) {  
  const indice = contatos.findIndex((contato) => contato.id === id)  
  
  if (indice === -1) {  
    console.log('Erro: Contato não encontrado!')  
    return false  
  }  
  // ... continua
```

O `findIndex` procura o contato pelo ID. Se devolver `-1`, o contato não existe: a função avisa e para.

atualizarContato: o novo e-mail está livre?

```
if (novosDados.email) {  
  for (let i = 0; i < contatos.length; i++) {  
    let atual = contatos[i]  
    if (atual.email === novosDados.email && atual.id !== id) {  
      console.log('Erro: O novo e-mail já está em uso!')  
      return false  
    }  
  }  
}
```

Só valida se o usuário digitou um e-mail novo. O `atual.id !== id` deixa a pessoa manter o próprio e-mail: só é erro se ele for de outro contato.

atualizarContato: só o que foi preenchido

```
contatos[indice].nome = novosDados.nome || contatos[indice].nome
contatos[indice].email = novosDados.email || contatos[indice].email

if (novosDados.telefones && novosDados.telefones.length > 0) {
  contatos[indice].telefones = novosDados.telefones
}
return true
}

export default atualizarContato
```

Enter vazio é texto vazio, que é falsy: o || mantém o valor antigo. Se vierem telefones novos, a lista antiga é substituída.



Delete: removendo um contato

removeContato

```
function removeContato(contatos, id) {  
  const indice = contatos.findIndex((contato) => contato.id === id)  
  
  if (indice === -1) {  
    console.log('Erro: Contato não encontrado!')  
    return false  
  }  
  
  contatos.splice(indice, 1)  
  console.log('Contato removido com sucesso!')  
  return true  
}
```

O mesmo `findIndex` da atualização acha a posição, e o `splice(indice, 1)` recorta exatamente aquele item.

Pedindo confirmação

```
case '4': {  
  let idRemover = parseInt(prompt('ID do contato a ser removido: '))  
  let confirmar = prompt('Tem certeza? (sim/nao): ')  
  
  if (confirmar.toLowerCase() === 'sim') {  
    removerContato(contatos, idRemover)  
  } else {  
    console.log('Operação cancelada.')  
  }  
  break  
}
```

O parseInt é essencial: o ID do objeto é número, e "3" === 3 é false. Sem converter, o findIndex nunca encontraria o contato.

Mais métodos que economizam laços

```
// some: algum item passa no teste?  
const existe = contatos.some((c) => c.email === email)  
  
// map: transforma cada item  
const nomes = contatos.map((c) => c.nome)  
// ['Ana Silva', 'Nicolas Motta']  
  
// reduce: junta tudo num valor só  
const total = contatos.reduce((soma, c) => soma + c.id, 0)
```

O laço de validação do e-mail vira uma linha com some. Laço com for continua valendo: esses métodos só deixam a intenção mais clara.



O que é um ID?

A "impressão digital" de cada registro

Por que usar IDs?



Unicidade

Cada registro é único. Imagine achar alguém num país onde todos se chamam João ou Maria.



Identificação

Garante que estamos mexendo no dado certo, e não num parecido.



Relacionamento

IDs ligam dados de listas diferentes, como veremos na clínica.



Performance

Buscar por um número é muito rápido: os sistemas são otimizados para isso.

Por que o e-mail precisa ser único?



Integridade

Sem duplicados, não há registros conflitantes e confusos.



Experiência do usuário

Ninguém se perde com várias contas no mesmo e-mail.



Segurança

E-mails duplicados podem ser explorados para tomar contas.



Comunicação

Notificações chegam uma vez só, para a pessoa certa.

Conferindo os requisitos

Requisito	Onde está no código
ID sequencial	adicionarContato: último ID + 1
Nome, e-mail e lista de telefones	menu, case '2': while com push
E-mail duplicado bloqueado	laço em adicionarContato
Telefones como lista	listarContatos: join(' ')
Atualizar só o que foi preenchido	atualizarContato: operador
Novo e-mail não pode estar em uso	atualizarContato: atual.id !== id
Remover pelo ID, com confirmação	findIndex + splice; menu, case '4'



Relacionamento entre os dados

Entendendo os relacionamentos



Sistemas reais têm várias listas

O Uber tem motoristas, passageiros e corridas. O iFood tem restaurantes, clientes e pedidos.



Guardamos o ID, não os dados

Em vez de repetir o nome do médico em cada consulta, a consulta guarda só o `idMedico`.



O código faz o papel do garçom

Lê o ID na consulta, vai até a lista de médicos e volta com o nome.

Você já fez um 1 para muitos (1:N)

```
// 1 contato tem N telefones
contato.telefones = ['(11) 91234-5678']

contato.telefones.push('(11) 98765-4321')

console.log(contato.telefones)
// ['(11) 91234-5678', '(11) 98765-4321']
```

Em vez de criar telefone1, telefone2, telefone3..., um array guarda quantos telefones forem necessários.

O problema da clínica



Dois cadastros

Um CRUD de pacientes e um CRUD de médicos.



Nome repetido

Se a ficha da Maria guarda "Dra. Grey" e a médica muda de sobrenome, é preciso alterar a ficha de todos os pacientes dela.

Como o sistema sabe que a paciente Maria tem consulta com a Dra. Grey, sem repetir os dados?

Usando IDs para conectar as listas



Atualiza num lugar só

A médica mudou de sobrenome? O ID continua o mesmo: nenhuma consulta precisa mudar.



Menos memória

Guardar um número ocupa bem menos que repetir textos grandes.



Integridade

Só agendamos consultas para IDs que realmente existem.

Os dados da clínica

```
// dados.js
const medicos = [
  { id: 1, nome: 'Dr. House', especialidade: 'Diagnóstico' },
  { id: 2, nome: 'Dra. Grey', especialidade: 'Cirurgia' },
]
const pacientes = [
  { id: 1, nome: 'John Doe' },
  { id: 2, nome: 'Jane Smith' },
]
let consultas = [
  { id: 1, data: '2023-01-10', idMedico: 1, idPaciente: 1 },
  { id: 2, data: '2023-02-15', idMedico: 2, idPaciente: 1 },
  { id: 3, data: '2023-03-20', idMedico: 1, idPaciente: 2 },
]

export default { medicos, pacientes, consultas }
```

Cada consulta guarda o idMedico e o idPaciente: é assim que ela se liga a um médico e a um paciente.

Médico, paciente e consulta



Médico 1 : N

O Dr. House pode ter consultas com vários pacientes ao longo do tempo.



Paciente 1 : N

O John Doe pode ter consultas com médicos diferentes, em momentos diferentes.



Consulta

Liga um médico a um paciente, numa data.

O paciente de ID 1 chegou na recepção. Como varrer a lista de consultas e achar **todas as dele, com o nome de cada médico?**



A Busca com Parâmetros

Dois métodos de array novos: filter e find

.filter(): a peneira

```
let usuarios = [  
  { id: 1, nome: 'Ana', ativo: true },  
  { id: 2, nome: 'Pedro', ativo: false },  
  { id: 3, nome: 'Maria', ativo: true },  
]  
  
let ativos = usuarios.filter((usuario) => usuario.ativo === true)  
console.log(ativos)  
// [ { id: 1, nome: 'Ana', ... }, { id: 3, nome: 'Maria', ... } ]
```

Varre o array inteiro e devolve um novo array só com os itens que passaram no teste. Se nenhum passar, devolve [].

.find(): o primeiro que bater

```
let medicos = [  
  { id: 1, nome: 'Dr. House' },  
  { id: 2, nome: 'Dra. Grey' },  
]  
  
let medico = medicos.find((medico) => medico.id === 2)  
console.log(medico)  
// { id: 2, nome: 'Dra. Grey' }
```

Para assim que encontra e devolve o objeto inteiro. Se não encontrar nada, devolve undefined.

filter x find x findIndex

filter

Quando você quer vários.

devolve um array
não achou: []

find

Quando você quer um objeto.

devolve o objeto
não achou: **undefined**

findIndex

Quando você quer a posição.

devolve o índice
não achou: **-1**

Ex.: todas as consultas do paciente 1 → filter. O médico de ID 2 → find. Onde está o contato 3, para remover → findIndex.

Funções auxiliares: médico e paciente

```
import dados from './dados.js'

function buscarMedicoPorId(idMedico) {
  return dados.medicos.find((medico) => medico.id === idMedico)
}

function buscarPacientePorId(idPaciente) {
  return dados.pacientes.find((paciente) => paciente.id === idPaciente)
}
```

Cada uma recebe um ID e devolve o objeto completo. No repositório, cada função fica no seu próprio arquivo.

imprimirConsulta: juntando as três listas

```
function imprimirConsulta(consulta) {  
  const medico = buscarMedicoPorId(consulta.idMedico)  
  const paciente = buscarPacientePorId(consulta.idPaciente)  
  
  let nomeMedico = 'Desconhecido'  
  if (medico) nomeMedico = medico.nome  
  
  let nomePaciente = 'Desconhecido'  
  if (paciente) nomePaciente = paciente.nome  
  
  console.log(`- ${consulta.data}: ${nomeMedico} com ${nomePaciente}`)  
}
```

A consulta só tem os IDs: o find busca o médico e o paciente. O if protege o caso de o ID não existir.

listarConsultasDoPaciente

```
function listarConsultasDoPaciente(idPaciente) {  
  const doPaciente = dados.consultas.filter(  
    (consulta) => consulta.idPaciente === idPaciente  
  )  
  
  if (doPaciente.length === 0) {  
    console.log('Nenhuma consulta agendada para este paciente.')  
    return  
  }  
  
  console.log('--- Consultas do Paciente ---')  
  doPaciente.forEach(imprimirConsulta)  
}
```

O filter separa as consultas do paciente e o forEach chama imprimirConsulta para cada uma delas.

O fluxo da busca

Ao chamar `listarConsultasDoPaciente(1)`, o sistema passa por três listas:



1. Filtro

Separa as consultas com `idPaciente: 1`.



2. Laço

O `forEach` passa por cada consulta encontrada.



3. Médico

`find` pelo `idMedico`.



4. Paciente

`find` pelo `idPaciente`.

Com os nomes em mãos, a consulta é impressa, e o laço segue para a próxima.

Como fica a saída

Resultado de `listarConsultasDoPaciente(1)`:

```
--- Consultas do Paciente ---  
- 2023-01-10: Dr. House com John Doe  
- 2023-02-15: Dra. Grey com John Doe
```

Duas consultas do mesmo paciente, com médicos diferentes: é o relacionamento 1:N funcionando.



Cuidado: o find pode devolver undefined

Se o ID não existir, o `find` devolve `undefined`, e acessar `medico.nome` quebra o programa.

- Por isso o código testa se o médico existe antes de usar, e mostra "Desconhecido" no lugar.
- Sempre que usar `find`, pense: e se não encontrar?

```
const medico = buscarMedicoPorId(99)    // undefined
console.log(medico.nome)
// TypeError: Cannot read properties of undefined (reading 'nome')
```

Hoje os dados **morrem quando
fechamos o terminal. Mais para a
frente no curso, vamos salvar tudo
num banco de dados de verdade!**

Atividade

CRUD

1. Termine o exemplo da agenda de contatos visto em sala.
2. Confira cada requisito da tabela "Conferindo os requisitos".
3. Faça a lista `codigo_crud_arrays.md` (push, find, filter e `findIndex`).
4. Escolha uma lista aplicada: **Agendamento de Consultas** ou **Flashcards**.



Dica: tudo está no repositório, em `modulo06/lista_de_exercicios/`. Use um contador para os IDs e padronize os e-mails com `toLowerCase()`.

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo06](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo06)



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 7, esse CRUD vira uma API de verdade.